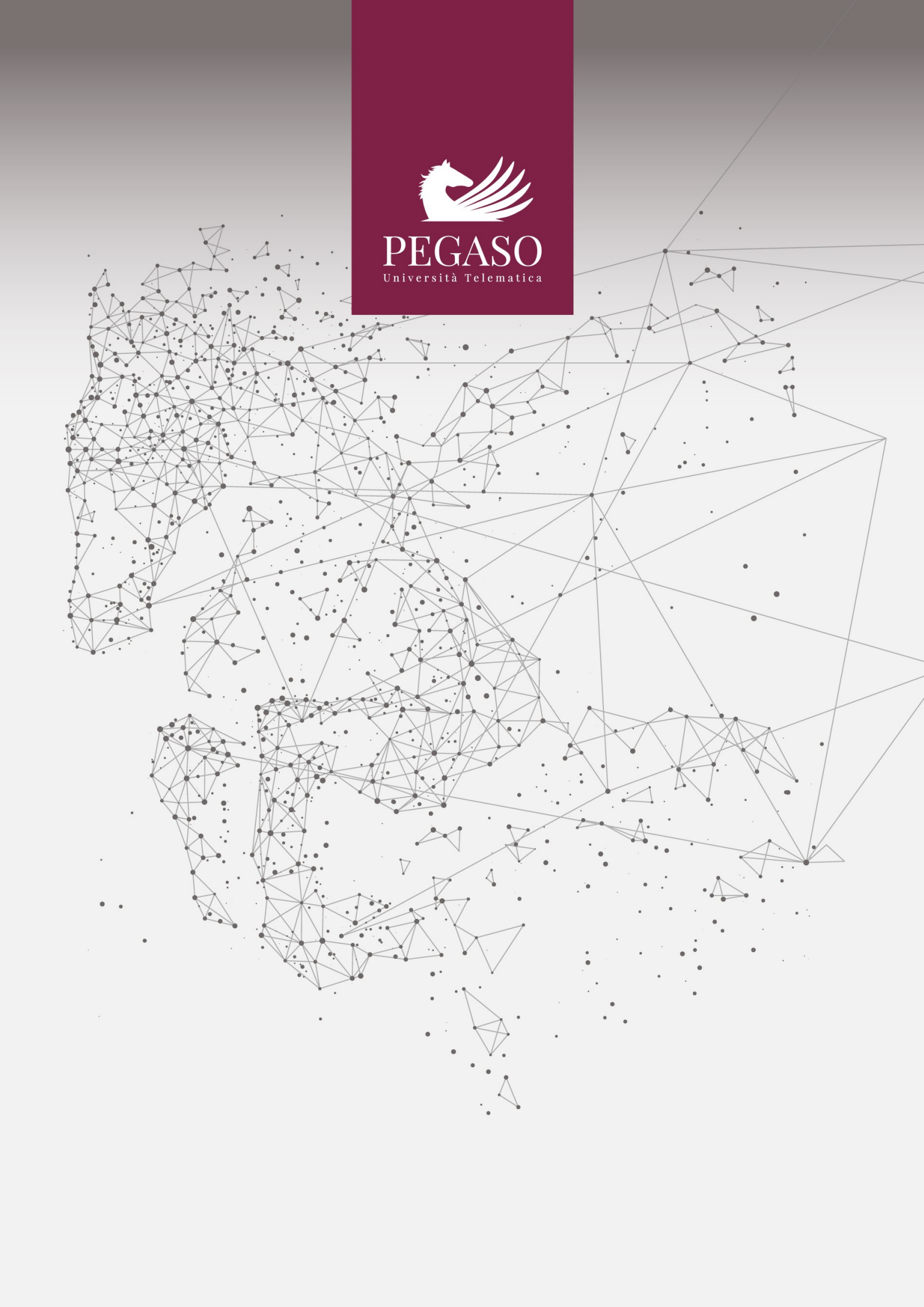

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. EREDITARIETÀ E POLIMORFISMO....4
3. EREDITARIETÀ E POLIMORFISMO: PROGETTAZIONE....6
4. EREDITARIETÀ E POLIMORFISMO: FORMALIZZAZIONE....10
5. EREDITARIETÀ E POLIMORFISMO: SIMULAZIONE....20
BIBLIOGRAFIA....25

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, viene presentata un'esercitazione di approfondimento su concetti e tecniche finora affrontati nel contesto delle classi di oggetti in linguaggio Python. Attraverso l'analisi, la progettazione ed, infine, la formalizzazione del problema oggetto di questa esercitazione verranno sperimentate le potenzialità dei meccanismi di *Ereditarietà* e *Polimorfismo* due tecniche molto importanti nell'ambito OOP. Nel primo paragrafo viene illustrato un tipico problema di classificazione del mondo animale costituito da una gerarchia di 5 classi: *MammiferoCarnivoro*, *Felino*, *Canide*, *Gatto* e *Cane*. Nel secondo si illustra prima la modellazione delle classi in termini di attributi e metodi per poi descriverle graficamente tramite il diagramma UML delle classi anche i legami strutturali.

Nel terzo paragrafo, infine, viene illustrata la formalizzazione in codice Python della gerarchia delle classi più un breve *main()* che ci permette di creare gli oggetti e attivare i metodi per verificare i meccanismi di '*ereditarietà*' e '*polimorfismo*'. Nell'ultimo paragrafo sono illustrate alcune sessioni di esecuzione del programma realizzato con i relativi risultati così da verificarne i risultati attesi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Ereditarietà e polimorfismo

Il meccanismo dell'ereditarietà è particolarmente vantaggioso in un contesto dove si deve risolvere un problema attraverso la definizione di una gerarchia di classi. L'*ereditarietà*, infatti, permette sia di riutilizzare, ereditandolo implicitamente, il codice già sviluppato in una classe base, sia di aggiungere solo il codice specifico della classe derivata.

Il meccanismo del *polimorfismo*, a sua volta, è strettamente legato all'ereditarietà tra classi e permette di ottenere comportamenti e risultati diversi richiamando metodi con lo stesso nome su oggetti diversi all'interno di una stessa gerarchia.

Le classi derivate (o specializzate) possono, quindi, sovrascrivere (*overriding*) i metodi della classe madre se si vogliono ottenere comportamenti e risultati diversi. I metodi sovrascritti avranno lo stesso nome sia nella classe base che in quella derivata. Inoltre, nella classe derivata è possibile comunque attivare, se necessario, il metodo corrispondente della classe-madre anche se il metodo è stato sovrascritto nella classe derivata. Per attivare il metodo definito nella classe base basta semplicemente premettere il nome della classe base al nome del metodo:

NomeClasseBase.NomeMetodo(self, argomenti).

Un tipico contesto in cui i meccanismi dell'*ereditarietà* e del *polimorfismo* sono facilmente comprensibili è quello della classificazione del mondo animale. In questa esercitazione, verrà affrontato il problema della creazione di oggetti software che simulano una gerarchia del regno animale. La gerarchia oggetto di questa esercitazione è:

1. MammiferoCarnivoro
    a) Felino
        i. Gatto
    b) Canide
        i. Cane

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Le principali caratteristiche possedute da questi animali sono:

1) *Speranza di vita media*: 1-20 anni
2) *Carattere*: Affettuoso, Tranquillo …
3) *Tipo di pelo*: Corto, Lungo, Raso, Duro
4) *Colore del Pelo*: Nero, Bianco, Rosso, Sabbia
5) *Taglia*: Piccola, Media, Grande, Molto grande
6) *Razza*: Alano, Akita, ..., Balinese, Cinerino, ...
7) *Comunicazione*: Metodo di comunicazione dell'animale

Ad esempio, di seguito si riporta la scheda a video che descrive le caratteristiche di un Gatto comprendente una descrizione della classe e una serie di attributi specifici dell'animale.

```
GATTO:  Il gatto domestico, la cui domesticazione iniziò in
        epoca piuttosto recente circa tra i 6.000 ed i 10.000,
        è il felide più diffuso al mondo

        Etá: 3
        speranza_vita: 15
        Carattere: Affettuoso
        Pelo: Corto
        Colore: Blu
        Razza: Certosino
        Habitat: Domestico
        Prede: Lucertole
        Nome: Grisella
Il Gatto comunica miagolando: Miaoo!
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Ereditarietà e polimorfismo: progettazione

In questo paragrafo si procederà con la modellazione delle classi di animali con oggetti software con attributi, metodi e relativi legami strutturali che simulano gli animali reali. Nel paragrafo precedente è stata presentata la gerarchia delle classi che alla radice vede la classe *MammiferoCarnivoro*, classe che avrà gli attributi che sono comuni a tutte le classi della gerarchia:

1) *__eta* età dell'animale
2) *__speranza_vita* aspettativa di vita in anni
3) *__carattere* carattere dell'animale
4) *__tipo_pelo* tipo di pelo
5) *__colore_pelo* colore del pelo
6) *__razza* razza dell'animale

e i seguenti metodi:

a) un metodo *get_AttributeName()* per ciascun attributo, che costituisce la sua interfaccia di lettura;
b) un metodo *set_AttributeName (aValue)* per ciascun attributo, che costituirà la sua interfaccia di modifica/scrittura;
c) *comunica()*, metodo che simula la modalità di interazione dell'animale. Questo metodo verrà utilizzato per sperimentare e comprendere meglio il meccanismo del *polimorfismo;*
d) *print_legenda()*, metodo che stampa a video la legenda della classe. Come per il metodo precedente, attraverso questo metodo si sperimenterà il meccanismo del *polimorfismo;*
e) *__str__()* riscrittura del metodo privato standard che restituisce una stringa per la rappresentazione a video degli oggetti della classe. È l'esempio più frequente di uso del meccanismo del *polimorfismo;*
f) *__init__(eta, vita, carattere, tipo_pelo, colore, razza)* metodo costruttore degli oggetti della classe.

La classe *Felino* è una classe derivata dalla precedente, che aggiunge i suoi attributi specifici a quelli ereditati dalla classe base:

1) *__habitat* habitat dell'animale
2) *__prede* le prede cacciate dall'animale

Tale classe si arricchisce con i seguenti metodi specifici:

a) i metodi get: *get_habitat() e get_prede();*
b) i metodi set: *set_habitat(aValue)* e *set_prede(aValue);*

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

c) *print_legenda()*, metodo che stampa a video la legenda della classe derivata. Stesso metodo della classe base, ma con comportamento diverso; per ottenerlo viene operato l'*override* del metodo base;

d) *__str__()* riscrittura del metodo privato standard che restituisce una stringa per la stampa a video della scheda informativa per gli oggetti della classe;

e) *__init__(eta, vita, carattere, tipo_pelo, colore, razza, habitat, prede)* metodo costruttore degli oggetti della classe *Felino.*

Per la classe *Canide* derivata dalla classe base *MammiferoCarnivoro* e parallela alla classe *Felino* nella gerarchia, si utilizzano i seguenti attributi specifici:

1) *__taglia* taglia dell'animale
2) *__origine* luogo d'origine dell'animale

oltre ai seguenti metodi propri:

a) i metodi get: *get_taglia() e get_origine()*

b) i metodi set: *set_taglia(aValue)* e *set_origine(aValue)*

c) *print_legenda()*, metodo che stampa a video la legenda della classe derivata. Stesso metodo della classe base, ma con comportamento diverso; per ottenerlo viene operato l'*override* del metodo base;

d) *__str__()* riscrittura del metodo privato standard che restituisce una stringa per la rappresentazione a video degli oggetti della classe;

e) *__init__(eta, vita, carattere, tipo_pelo, colore, razza, taglia, origine)* metodo costruttore degli oggetti della classe *Felino.*

La classe *Gatto* derivata dalla classe base *Felino*, presenterà i seguenti attributi specifici:

1) *__nome* nome proprio dell'animale

oltre ai seguenti metodi:

a) il metodo get: *get_nome();*

b) il metodo set: *set_nome(aValue)*;

c) *comunica()*, stesso metodo della classe base, ma con comportamento diverso; viene fatto l'*override* del metodo ereditato;

d) *print_legenda()*, metodo che stampa a video la legenda della classe *Gatto*. Stesso metodo della classe base, ma con comportamento diverso; per ottenerlo viene operato l'*override* del metodo base;

e) *__str__()* riscrittura del metodo privato standard che restituisce una stringa per la rappresentazione a video degli oggetti della classe;

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

f) *__init__(eta, vita, carattere, tipo_pelo, colore, razza, habitat, prede, nome)* metodo costruttore degli oggetti della classe.

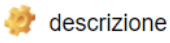
La classe *Cane* derivata dalla classe base *Canide*, avrà i seguenti attributi specifici:

1) *__nome* nome proprio dell'animale
2) *__utilizzato_per* ad es. Caccia, Compagnia, ….

oltre ai metodi aggiuntivi rispetto a quelli già ereditati:

a) i metodi get per gli attributi propri: *get_nome()* e *get_utilizzato_per ();*
b) i metodi set per gli attributi propri: *set_nome(aValue)* e *set_utilizzato_per (aValue);*
c) *comunica()*, in questo caso il comportamento dovrà essere diverso dal metodo ereditato (*polimorfismo*);
d) *print_legenda()*, metodo che stampa a video la legenda della classe *Cane*. Stesso metodo della classe base, ma con comportamento diverso; per ottenerlo viene operato l'*override* del metodo base;
e) *__str__()* riscrittura del metodo privato standard che restituisce una stringa per la rappresentazione a video degli oggetti della classe;
f) *__init__(eta, vita, carattere, tipo_pelo, colore, razza, taglia, origine, nome, utilizzato)* metodo costruttore degli oggetti della classe.

Prima di descrivere in forma grafica il diagramma UML delle classi, è opportuno chiarire meglio alcuni elementi strutturali.

Un attributo di classe (come nel caso dell'attributo descrizione), poiché esprime una caratteristica della classe e non di un oggetto/istanza, viene mantenuto a livello di classe. In maniera grafica nel diagramma è rappresentato come 

.

Il legame di specializzazione dalla classe derivata alla classe base, viene rappresentato graficamente con una freccia con la punta vuota, come riportato nella figura seguente.

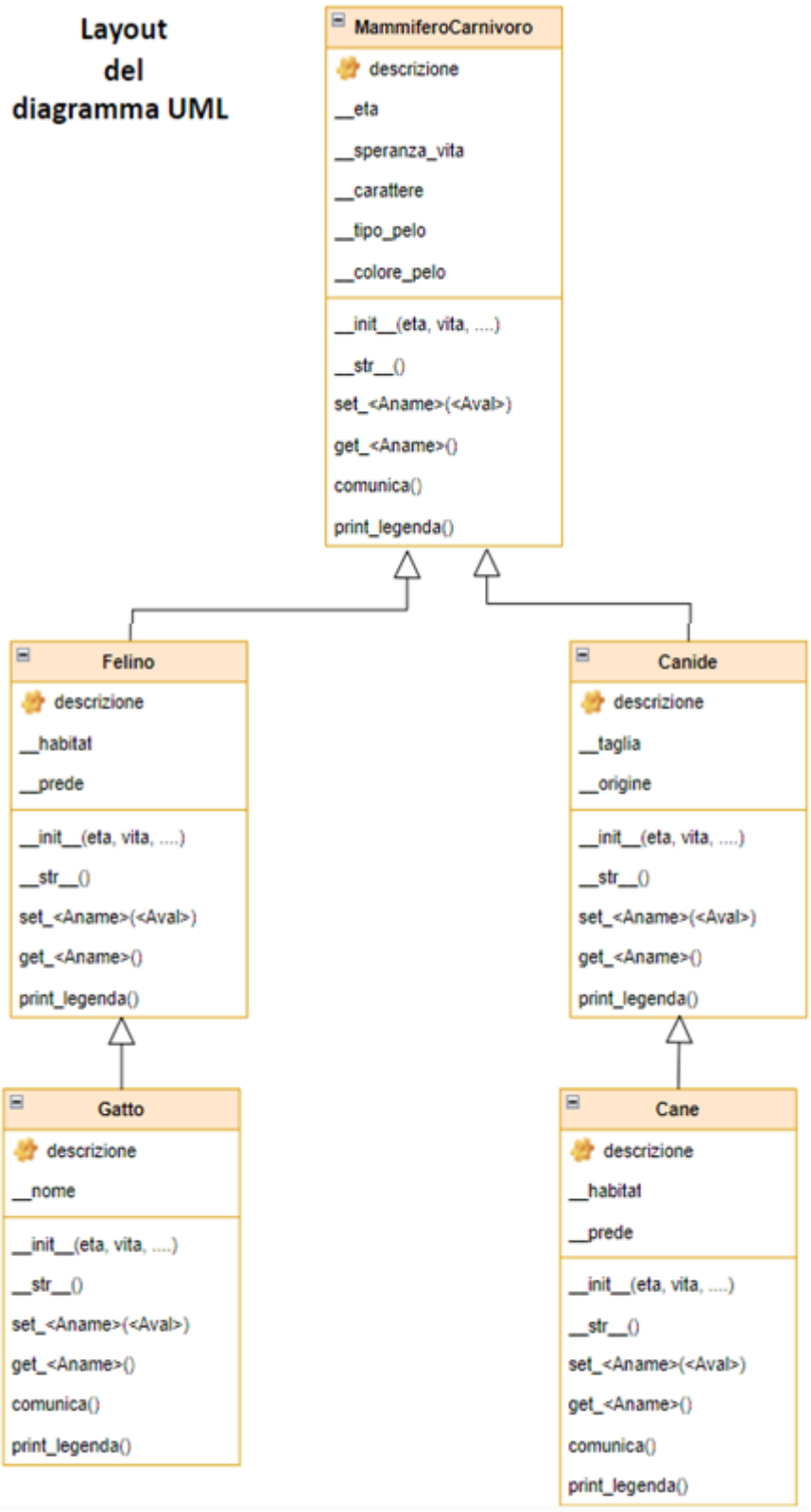


Nella figura seguente, invece, si riporta il layout del diagramma UML delle classi appena illustrate.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Layout del diagramma UML



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Ereditarietà e polimorfismo: formalizzazione

In questo paragrafo, completata la fase di progettazione, verrà illustrata la codifica prima delle classi e poi della funzione *main()* in linguaggio Python.

L'intera gerarchia delle classi verrà implementata in un unico modulo (*file*) denominato *GerarchiaClassi.py*. Per tutte le classi verranno seguite le seguenti modalità implementative/algoritmiche:

- l'attributo (descrizione) verrà implementato con una variabile pubblica a livello di classe. Viene creata nel momento in cui viene elaborata la definizione della classe. È possibile accedervi in qualunque momento in un contesto in cui è visibile la classe con la seguente sintassi: *NomeModulo.NomeClasse.descrizione;*
- tutti gli attributi di istanza/oggetto verranno implementati con delle variabili NON pubbliche. Il nome inizia sempre con il doppio trattino basso;
- il metodo costruttore *__init__()* nelle classi derivate richiama prima il metodo *__init__()* della classe-base e poi assegna/crea le proprie variabili (quelle ereditate sono già assegnate/create dalla chiamata al costruttore della classe-base);
- tutti i metodi *setter* assegnano il valore ricevuto come parametro alla variabile corrispondente;
- tutti i metodi *getter* ritornano semplicemente il valore della variabile corrispondente;
- il metodo *__str__()* viene riscritto in ciascuna classe per formattare opportunamente la visualizzazione a video di un oggetto. L'algoritmo concatena tutti gli attributi, premettendo il carattere «\n» e una descrizione del valore. L'algoritmo richiama prima il corrispondente metodo *__str__()* della classe-madre per ottenere la stringa degli attributi ereditati e poi vi concatena i propri gli attributi per costruire la stringa finale. Ritorna la stringa descrittiva dell'oggetto;
- il metodo *print_legenda()* stampa a video il valore della variabile *descrizione*. Mentre il metodo *comunica()* stampa a video il modo di comunicare dell'animale. Ambedue i metodi vengono definiti nella classe radice della gerarchia e poi ne viene effettuato l'*override* nelle sottoclassi dove è necessario un comportamento differente.

  Di seguito si riporta il codice Python della classe *MammiferoCarnivoro*.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
class MammiferoCarnivoro:
    #docstring della classe
    """Classe dei MAMMIFERI CARNIVORI"""
    descrizione="\nMAMMIFERI CARNIVORI:\tI mammiferi carnivori sono animali provvisti \
                "\n\tdi denti"+particolari, utili al consumo della carne di altri animali;"+\
                "\n\tdenti canini che si usano per catturare la preda e per strappare"+\
                "\n\tla carne."

    def __init__(self, eta, vita, carattere, tipo_pelo, colore, razza):
        self.__eta = eta
        self.__speranza_vita = vita
        self.__carattere = carattere
        self.__tipo_pelo = tipo_pelo
        self.__colore_pelo = colore
        self.__razza = razza

    def get_eta(self):
        return self.__eta

    def set_eta(self, eta):
        self.__eta = eta

    def get_speranza_vita(self):
        return self.__speranza_vita

    def set_speranza_vita(self, vita):
        self.__speranza_vita = vita

    def get_carattere(self):
        return self.__carattere

    def set_carattere(self, carattere):
        self.__scarattere = carattere

    def get_tipo_pelo(self):
        return self.__tipo_pelo

    def set_tipo_pelo(self, tipo_pelo):
        self.__tipo_pelo = tipo_pelo

    def get_colore_pelo(self):
        return self.__colore_pelo

    def set_colore_pelo(self, colore):
        self.__colore_pelo = colore

    def get_razza(self):
        return self.__razza

    def set_razza(self, razza):
        self.__razza = razza

    def print_legenda(self):
        print(MammiferoCarnivoro.descrizione)

    def comunica(self):
        #raise NotImplementedError("Metodo NON implementato!")
        print("\n>>>La comunicazione dipende dall'animale!")

    def __str__(self):
        return "\n\tEtá: "+str(self.__eta)+\
               "\n\tsperanza_vita: "+str(self.__speranza_vita)+\
               "\n\tCarattere: "+self.__carattere+\
               "\n\tPelo: "+self.__tipo_pelo+\
               "\n\tColore: "+self.__colore_pelo+\
               "\n\tRazza: "+self.__razza
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Di seguito si riporta il codice Python della classe *Felino*.

```
class Felino(MammiferoCarnivoro):
    #docstring della classe
    """Classe dei FELINI"""
    descrizione="\nFELINI:\tI Felini sono caratterizzati da agilità nella caccia,"+\
                "\n\tche include specie molto grandi."+\
                "\n\tEssi includono leoni e tigri,ma anche animali più piccoli"+\
                "\n\tcome i gatti comuni."

    def __init__(self, eta, vita, carattere, tipo_pelo, colore, razza, habitat, prede):
        #super().__init__(self, eta, vita, carattere, tipo_pelo, colore, razza)
        MammiferoCarnivoro.__init__(self, eta, vita, carattere, tipo_pelo, colore, razza)
        self.__habitat = habitat
        self.__prede = prede

    def get_habitat(self):
        return self.__habitat

    def set_habitat(self, habitat):
        self.__habitat = habitat

    def get_prede(self):
        return self.__prede

    def set_prede(self, pelo):
        self.__prede = prede

    def print_legenda(self):
        print(Felino.descrizione)

    def __str__(self):
        return MammiferoCarnivoro.__str__(self)+\
                "\n\tHabitat: "+self.__habitat+\
                "\n\tPrede: "+ self.__prede
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Di seguito si riporta il codice Python della classe *Canide*.

```
class Canide(MammiferoCarnivoro):
    #docstring della classe
    """Classe dei CANIDI"""
    descrizione="\nCANIDI:\tLa famiglia dei Canidi comprende Carnivori di norma"+\
                "\n\tcon 5 dita negli arti anteriori e quattro in quelli"+\
                "\n\tposteriori, con piedi digitigradi, unghie non retrattili"+\
                "\n\tatte a scavare, testa allungata: I denti canini sono ben"+\
                "\n\tsviluppati ed atti a dilaniare le prede"

    def __init__(self, eta, vita, carattere, tipo_pelo, colore, razza, taglia, origine):
        MammiferoCarnivoro.__init__(self, eta, vita, carattere, tipo_pelo, colore, razza)
        self.__taglia = taglia
        self.__origine = origine

    def get_taglia(self):
        return self.__taglia

    def set_taglia(self, taglia):
        self.__taglia = taglia

    def get_origine(self):
        return self.__origine

    def set_origine(self, origine):
        self.__origine = origine

    def print_legenda(self):
        print(Canide.descrizione)

    def __str__(self):
        return MammiferoCarnivoro.__str__(self)+\
               "\n\tTaglia: "+ str(self.__taglia)+\
               "\n\tOrigine: "+ str(self.__origine)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Di seguito si riporta il codice Python della classe *Gatto*.

```
class Gatto(Felino):
    #docstring della classe
    """Classe dei GATTI"""
    descrizione="\nGATTO:\tIl gatto domestico, la cui domesticazione iniziò in"+\
                "\n\tepoca piuttosto recente circa tra i 6.000 ed i 10.000,"+\
                "\n\tè il felide più diffuso al mondo"
    def __init__(self, eta, vita, carattere, tipo_pelo, colore, razza, habitat, prede,nome):
        Felino.__init__(self, eta, vita, carattere, tipo_pelo, colore, razza, habitat, prede)
        self.__nome = nome

    def get_nome(self):
        return self.__nome

    def set_nome(self, nome):
        self.__nome = nome

    def get_prede(self):
        return self.__prede

    def set_prede(self, prede):
        self.__nome = prede

    def print_legenda(self):
        print(Gatto.descrizione)

    def comunica(self):
        print("Il Gatto comunica miagolando: Miaoo!")

    def __str__(self):
        return Felino.__str__(self)+\
               "\n\tNome: "+ self.__nome
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Di seguito si riporta il codice Python della classe *Cane*.

```
class Cane(Canide):
    #docstring della classe
    """Classe dei CANI"""
    descrizione="\nCANE:\tIl passaggio dal lupo al cane che vediamo oggi,"+\
                "\n\tha richiesto circa 5.000 generazioni: il cane è stato"+\
                "\n\til primo animale ad essere addomesticato"

    def __init__(self, eta, vita, carattere, tipo_pelo, colore, razza, taglia, origine,nome,utilizzato):
        Canide.__init__(self, eta, vita, carattere, tipo_pelo, colore, razza, taglia, origine)
        self.__nome = nome
        self.__utilizzato_per = utilizzato

    def get_nome(self):
        return self.__nome

    def set_nome(self, nome):
        self.__nome = nome

    def get_utilizzato_per(self):
        return self.__utilizzato_per

    def set_utilizzato_per(self, utilizzato):
        self.__utilizzato_per = utilizzato

    def print_legenda(self):
        print(Cane.descrizione)

    def comunica(self):
        print("Il Cane comunica abbaiando/ululando: Bauu/Uuh")

    def __str__(self):
        return Canide.__str__(self)+\
               "\n\tNome: "+ self.__nome+\
               "\n\tUtilizzato per: "+ self.__utilizzato_per
```

Di seguito viene descritto il codice delle funzioni ausiliari di supporto alla funzione main() per la simulazione del programma:

- *display_menu()* - visualizza a schermo le opzioni (con descrizione) del menu di funzionalità attivabili. L'algoritmo realizza la funzione mediante una serie di chiamate della funzione *print()* come mostrato nel codice seguente;

```
def display_menu():
    print()
    print("MENU")
    print("1) Crea un Mammifero Carnivoro")
    print("2) Crea un Felino")
    print("3) Crea un Canide")
    print("4) Crea un Gatto")
    print("5) Crea un Cane")
    print("6) Termina il programma")
    print()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- *chiedi_int(prompt)* - funzione per richiedere l'input di un valore numerico intero. Riceve un parametro - il messaggio di invito da visualizzare – e restituisce il valore immesso dall'utente. L'algoritmo implementato itera la richiesta finché non viene digitato un valore corretto. Il controllo viene effettuato catturando l'eventuale eccezione prodotta dal casting della stringa digitata dall'utente. Di seguito si riporta il codice Python della funzione chiedi_int();

```
def chiedi_int(prompt):
    valido = False
    while not valido:
        try:
            n = int(input(prompt))
            valido = True
        except:
            print("\tERRORE! Il valore inserito non è valido")
    return n
```

- *chiedi_info_base()* - funzione per richiedere l'input dei dati per creare un oggetto di classe MammiferoCarnivoro che non riceve parametri e restituisce i sei valori immessi dall'utente. L'algoritmo utilizzato esegue in sequenza delle chiamate alle funzioni *chiedi_int()* e *input()* come mostrato nel codice Python che segue;

```
def chiedi_info_base():
    #Informazioni:  eta, vita, carattere, tipo_pelo, colore, razza
    e = chiedi_int("Inserire l'età: ")
    v = chiedi_int("Inserire la speranza di vita in anni: ")
    ca = input("Inserire il Carattere: ")
    p = input("Inserire il Tipo di Pelo: ")
    col = input("Inserire il Colore del Pelo: ")
    r = input("Inserire la Razza: ")

    return e,v,ca,p,col,r
```

- le funzioni *chiedi_info_felino()* e *chiedi_info_canide()* – funzioni per richiedere l'input dei dati specifici della classe. Esse non ricevono parametri e restituiscono i due valori immessi dall'utente. L'algoritmo esegue due chiamate successive della funzione *input* e ritorna i valori memorizzati nelle variabili come mostrato nel codice Python seguente;

PEGASO
Università Telematica

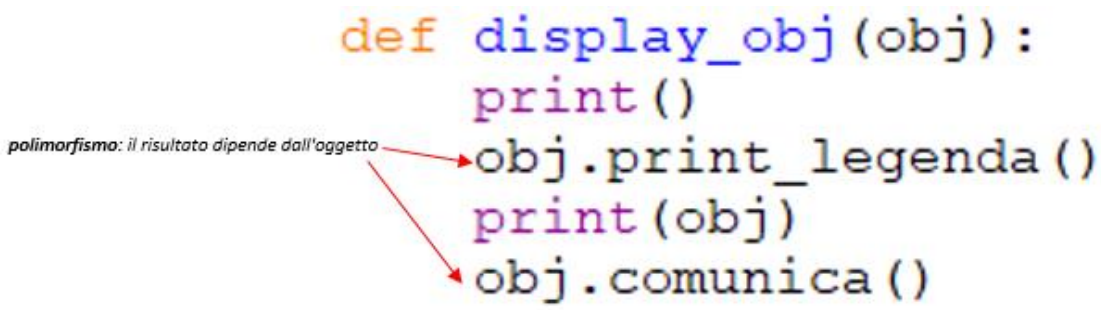
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def chiedi_info_felino():
    #Informazioni:  habitat, prede
    h = input("Inserire l'abitat: ")
    pr = input("Inserire le prede: ")

    return h,pr
def chiedi_info_canide():
    #Informazioni:  taglia, origine
    t = input("Inserire la taglia: ")
    o = input("Inserire l'origine: ")

    return t,o
```

- *display_obj()* - funzione per visualizzare a video i dati specifici di un oggetto tra quelli che è consentito creare. Essa non riceve parametri e non restituisce valori. L'algoritmo implementato esegue in sequenza delle chiamate alle funzioni di interfaccia dell'oggetto e alla funzione *print()* come mostrato nel codice Python seguente.



Le costanti riportate nel codice seguente, invece, sono state definite per migliorare la leggibilità e per semplificare gli algoritmi implementati.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import GerarchiaClassi

#Costanti globali utilizzate
UN_MAMMIFEROCARNIVORO = 1
UN_FELINO = 2
UN_CANIDE = 3
UN_GATTO = 4
UN_CANE = 5
QUIT_CHOICE = 6
```

Infine, la funzione *main()* crea su richiesta gli oggetti e ne visualizza a video la scheda informativa da cui si evincono le caratteristiche (attributi) e il comportamento (metodi) dell'oggetto. L'algoritmo implementato è formato essenzialmente da un ciclo iterativo che viene eseguito finché l'utente non sceglie di terminare il programma. Nel ciclo vengono intraprese le seguenti azioni:

1. visualizzazione del menu;
2. richiesta di effettuare una scelta;
3. richiesta di tutti i dati necessari per creare l'oggetto scelto;
4. creazione dell'oggetto;
5. visualizzazione del risultato.

Di seguito si riporta il codice Python della funzione main().

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def main():
    choice = 0
    obj = 0
    while choice != QUIT_CHOICE:
        display_menu()

        choice = int(input('Inserire la scelta: '))
        if choice in [UN_MAMMIFEROCARNIVORO,UN_FELINO,UN_CANIDE,UN_GATTO,UN_CANE]:
            # eta, vita, carattere, tipo_pelo, colore, razza
            e,v,ca,p,col,r = chiedi_info_base()

        if choice == UN_MAMMIFEROCARNIVORO:
            obj = GerarchiaClassi.MammiferoCarnivoro(e,v,ca,p,col,r)
            display_obj(obj)

        elif choice == UN_FELINO:
            # habitat, prede
            hab,prede = chiedi_info_felino()
            obj = GerarchiaClassi.Felino(e,v,ca,p,col,r,hab,prede)
            display_obj(obj)

        elif choice == UN_CANIDE:
            # taglia, origine
            t,o = chiedi_info_canide()
            obj = GerarchiaClassi.Canide(e,v,ca,p,col,r,t,o)
            display_obj(obj)

        elif choice == UN_GATTO:
            # habitat, prede
            hab,prede = chiedi_info_felino()
            nome = input("Inserire il nome del Gatto: ")
            obj = GerarchiaClassi.Gatto(e,v,ca,p,col,r,hab,prede,nome)
            display_obj(obj)

        elif choice == UN_CANE:
            # nome, utilizzato_per
            t,o = chiedi_info_canide()
            nome = input("Inserire il nome del Cane: ")
            u = input("Inserire l'utilizzo del Cane: ")
            obj = GerarchiaClassi.Cane(e,v,ca,p,col,r,t,o,nome,u)
            display_obj(obj)

        elif choice!= QUIT_CHOICE:
            print('ERRORE: scelta non valida')


    print("Programma TERMINATO!")
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Ereditarietà e polimorfismo: simulazione

In questo paragrafo vengono illustrate alcune sessioni di esecuzione del programma che simulano la creazione degli oggetti su richiesta. Dai valori delle schede visualizzate a video si comprendono le caratteristiche dell'oggetto ed il suo comportamento.

La prima è una richiesta di creazione di un oggetto/animale della classe *MammiferoCarnivoro*.

```
MENU
1) Crea un Mammifero Carnivoro
2) Crea un Felino
3) Crea un Canide
4) Crea un Gatto
5) Crea un Cane
6) Termina il programma

Inserire la scelta: 1
Inserire l'età: 1
Inserire la speranza di vita in anni: 100
Inserire il Carattere: Aggressivo
Inserire il Tipo di Pelo: Raso
Inserire il Colore del Pelo: Bianco
Inserire la Razza: Licaone
```

risultato del metodo print_legenda()

```
MAMMIFERI CARNIVORI:    I mammiferi carnivori sono animali provvisti
        di denti particolari, utili al consumo della carne di altri animali;
        denti canini che si usano per catturare la preda e per strappare
        la carne.

        Età: 1
        speranza_vita: 100
        Carattere: Aggressivo
        Pelo: Raso
        Colore: Bianco
        Razza: Licaone
```

risultato del metodo comunica()

```
>>>La comunicazione dipende dall'animale!
```

Come è possibile facilmente verificare dalla scheda informativa (elenco degli attributi), l'oggetto creato ha tutti e soli gli attributi della classe scelta. Il metodo *print_legenda()* visualizza la legenda della classe *MammiferoCarnivoro* ed anche il metodo *comunica()* visualizza la stringa generica e non il verso come nel caso di *Gatto* e *Cane.*

La seconda è la richiesta di creazione di un oggetto/animale della classe Felino.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
MENU
1) Crea un Mammifero Carnivoro
2) Crea un Felino
3) Crea un Canide
4) Crea un Gatto
5) Crea un Cane
6) Termina il programma

Inserire la scelta: 2
Inserire l'età: 1
Inserire la speranza di vita in anni: 15
Inserire il Carattere: Cacciatore
Inserire il Tipo di Pelo: Corto
Inserire il Colore del Pelo: Grigio
Inserire la Razza: Tasso
Inserire l'abitat: Prateria
Inserire le prede: Piccoli rettili
```

risultato del metodo *print_legenda()*

```
FELINI: I Felini sono caratterizzati da agilità nella caccia,
        che include specie molto grandi.
        Essi includono leoni e tigri,ma anche animali più piccoli
        come i gatti comuni.

        Etá: 1
        speranza_vita: 15
        Carattere: Cacciatore
        Pelo: Corto
        Colore: Grigio
        Razza: Tasso
        Habitat: Prateria
        Prede: Piccoli rettili
```

risultato del metodo *comunica()*

```
>>>La comunicazione dipende dall'animale!
```

Anche per questa esecuzione la scheda informativa visualizzata è quella di un oggetto della classe richiesta *Felino*. Il risultato della esecuzione dei metodi *print_legenda()* e *comunica()* è quello atteso.

Si noti che il metodo *comunica()* non esiste nella classe *Felino*, ma viene ereditato dalla classe madre perché il risultato della funzione deve essere lo stesso.

La terza è la richiesta di creazione di un oggetto/animale della classe *Canide*.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
MENU
1) Crea un Mammifero Carnivoro
2) Crea un Felino
3) Crea un Canide
4) Crea un Gatto
5) Crea un Cane
6) Termina il programma

Inserire la scelta: 3
Inserire l'età: 2
Inserire la speranza di vita in anni: 17
Inserire il Carattere: Indipendente
Inserire il Tipo di Pelo: Lungo
Inserire il Colore del Pelo: Nero
Inserire la Razza: Lupo Grigio
Inserire la taglia: Media
Inserire l'origine: Nord America
```

risultato del metodo *print_legenda()*

```
CANIDI: La famiglia dei Canidi comprende Carnivori di norma
        con 5 dita negli arti anteriori e quattro in quelli
        posteriori, con piedi digitigradi, unghie non retrattili
        atte a scavare, testa allungata: I denti canini sono ben
        sviluppati ed atti a dilaniare le prede

        Etá: 2
        speranza_vita: 17
        Carattere: Indipendente
        Pelo: Lungo
        Colore: Nero
        Razza: Lupo Grigio
        Taglia: Media
        Origine: Nord America
```

risultato del metodo *comunica()*

```
>>>La comunicazione dipende dall'animale!
```

La scheda informativa visualizza in questo caso gli attributi *Taglia* e *Origine* che sono specifici della classe *Canide*. Per i risultati dei metodi *print_legenda()* e *comunica()* valgono le stesse considerazioni della classe *Felino.*

La quarta è la richiesta di creazione di un oggetto/animale della classe *Gatto.*

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
MENU
1) Crea un Mammifero Carnivoro
2) Crea un Felino
3) Crea un Canide
4) Crea un Gatto
5) Crea un Cane
6) Termina il programma

Inserire la scelta: 4
Inserire l'età: 3
Inserire la speranza di vita in anni: 15
Inserire il Carattere: Affettuoso
Inserire il Tipo di Pelo: Corto
Inserire il Colore del Pelo: Blu
Inserire la Razza: Certosino
Inserire l'abitat: Domestico
Inserire le prede: Lucertole
Inserire il nome del Gatto: Grisella
```

risultato del metodo print_legenda()

```
GATTO:  Il gatto domestico, la cui domesticazione iniziò in
        epoca piuttosto recente circa tra i 6.000 ed i 10.000,
        è il felide più diffuso al mondo

        Etá: 3
        speranza_vita: 15
        Carattere: Affettuoso
        Pelo: Corto
        Colore: Blu
        Razza: Certosino
        Habitat: Domestico
        Prede: Lucertole
        Nome: Grisella
```

risultato del metodo comunica()

```
Il Gatto comunica miagolando: Miaoo!
```

La scheda informativa in questo caso visualizza l'attributo Nome che è specifico della classe Gatto. Il metodo *print_legenda*() visualizza la legenda descrittiva del gatto domestico ed anche *comunica*() visualizza proprio il modo di comunicare di un oggetto di classe *Gatto* (il metodo ha avuto un comportamento diverso e conforme all'oggetto creato).

La quinta è la richiesta di creazione di un oggetto/animale della classe *Cane*.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
MENU
1) Crea un Mammifero Carnivoro
2) Crea un Felino
3) Crea un Canide
4) Crea un Gatto
5) Crea un Cane
6) Termina il programma

Inserire la scelta: 5
Inserire l'età: 4
Inserire la speranza di vita in anni: 10
Inserire il Carattere: Giocarellone
Inserire il Tipo di Pelo: Raso
Inserire il Colore del Pelo: Rosso
Inserire la Razza: Alano
Inserire la taglia: Molto grande
Inserire l'origine: Germania
Inserire il nome del Cane: Sansone
Inserire l'utilizzo del Cane: Compagnia
```

risultato del metodo *print_label()*

```
CANE:   Il passaggio dal lupo al cane che vediamo oggi,
        ha richiesto circa 5.000 generazioni: il cane è stato
        il primo animale ad essere addomesticato

        Etá: 4
        speranza_vita: 10
        Carattere: Giocarellone
        Pelo: Raso
        Colore: Rosso
        Razza: Alano
        Taglia: Molto grande
        Origine: Germania
        Nome: Sansone
        Utilizzato per: Compagnia
```

risultato del metodo *comunica()*

```
Il Cane comunica abbaiando/ululando: Bauu/Uuh
```

La scheda informativa visualizza gli attributi che sono specifici della classe *Cane* (Nome e Utilizzato per). I metodi *print_legenda*() e *comunica*() hanno il comportamento specifico di un oggetto di classe Cane e non quello della classe madre.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).